

Aerofit Project



About Aerofit

Aerofit is a popular Indian fitness equipment brand known for providing affordable and reliable workout machines for home gyms and small fitness centers. The brand focuses on improving fitness accessibility by offering easy-to-use, durable machines.

Introduction

Aerofit is a fitness equipment company offering different treadmill models for customers with varying needs, budgets, and fitness levels. To better understand customer buying behavior and usage patterns, a detailed analysis of the dataset was performed. Aerofit improve its product strategy, customer targeting, marketing approach, and overall business performance.

Business Problem

Aerofit faces several business challenges in today’s fitness market. Strong competition from modern fitness brands makes it difficult to attract new customers. Many of Aerofit’s machines lack advanced features like app connectivity, which reduces their appeal. After the pandemic, the demand for home fitness equipment dropped, leading to lower sales. The brand also struggles with weak digital marketing, slow service and maintenance support, and pressure to keep prices low, which limits innovation. In addition, Aerofit has a small presence in commercial gyms, reducing its overall growth opportunities.

Step-1: Dataset

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

In [2]: data=pd.read_csv("/content/aerofit_treadmill.txt")

In [3]: data.head()

Out[3]:
```

	Product	Age	Gender	Education	MaritalStatus	Usage	Fitness	Income	Miles
0	KP281	18	Male	14	Single	3	4	29562	112
1	KP281	19	Male	15	Single	2	3	31836	75
2	KP281	19	Female	14	Partnered	4	3	30699	66
3	KP281	19	Male	12	Single	3	3	32973	85
4	KP281	20	Male	13	Partnered	4	2	35247	47

```
In [4]: def price(product):
    if product=='KP281':
        return("$1,500")
    elif product=='KP481':
        return("$1,750")
    else:
        return("$2500")

In [5]: data['price_product']=data['Product'].apply(price)
```

```
In [6]: data.shape
```

Out[6]: (180, 10)

```
In [7]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Product               180 non-null   object
1   Age                   180 non-null   int64
2   Gender                180 non-null   object
3   Education              180 non-null   int64
4   MaritalStatus         180 non-null   object
5   Usage                 180 non-null   int64
6   Fitness               180 non-null   int64
7   Income                180 non-null   int64
8   Miles                 180 non-null   int64
9   price_product         180 non-null   object
dtypes: int64(6), object(4)
memory usage: 14.2+ KB
```

```
In [8]: data['price_product'] = data['price_product'].astype(str).str.replace('$', '', regex=False).str.replace(',', '', regex=False).
```

Step-2: Data_Cleaning

```
In [9]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Product               180 non-null   object
1   Age                   180 non-null   int64
2   Gender                180 non-null   object
3   Education              180 non-null   int64
4   MaritalStatus         180 non-null   object
5   Usage                 180 non-null   int64
6   Fitness               180 non-null   int64
7   Income                180 non-null   int64
8   Miles                 180 non-null   int64
9   price_product         180 non-null   int64
dtypes: int64(7), object(3)
memory usage: 14.2+ KB
```

```
In [10]: data.isnull().sum()
```

Out[10]:

	0
Product	0
Age	0
Gender	0
Education	0
MaritalStatus	0
Usage	0
Fitness	0
Income	0
Miles	0
price_product	0

dtype: int64

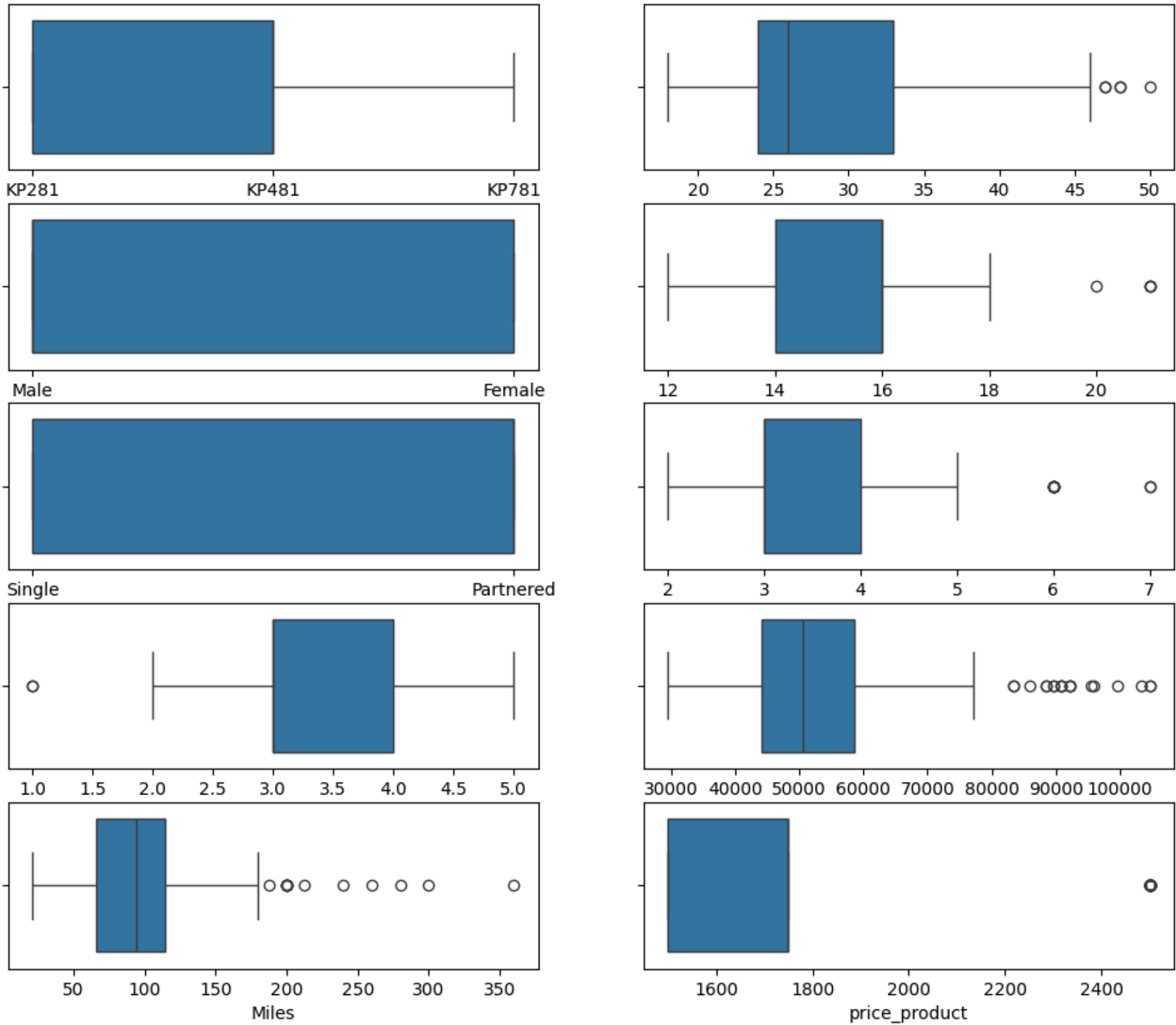
Step-3: Detect Outliers:

```
In [11]: data_num=data.columns
data_num
```

Out[11]: Index(['Product', 'Age', 'Gender', 'Education', 'MaritalStatus', 'Usage', 'Fitness', 'Income', 'Miles', 'price_product'], dtype='object')

```
In [12]: fig,axes= plt.subplots(nrows=5,ncols=2,figsize=(12,10))
axes=axes.flatten()
```

```
for i,j in enumerate(data_num):
    sns.boxplot(x=data[j],ax=axes[i])
```



In this Dataset the outliers are in Income and Miles

Step-4: Remove Outliers

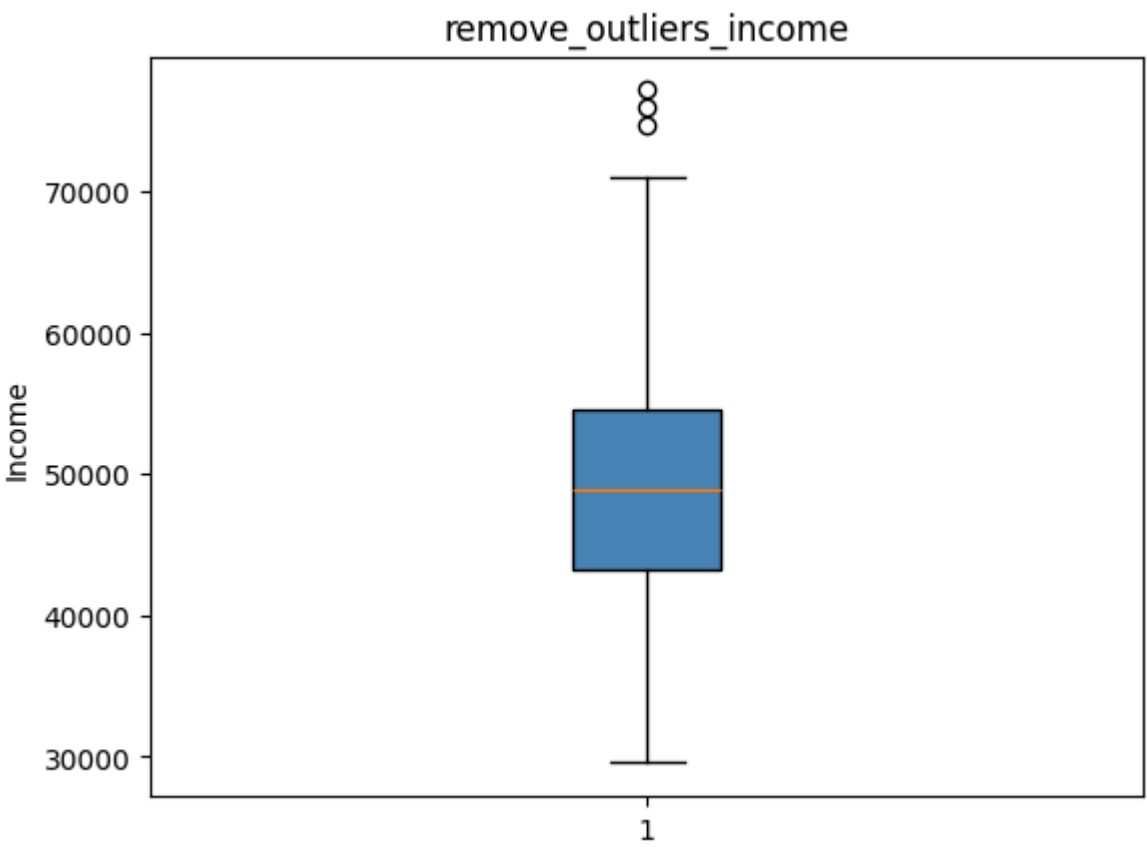
1.Income

```
In [13]: # Compute IQR (Interquartile Range)
Q1 = data['Income'].quantile(0.25)
Q3 = data['Income'].quantile(0.75)
IQR = Q3 - Q1

# Define acceptable range
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

# Remove outliers
data_no_outliers = data[(data['Income'] >= lower_limit) & (data['Income'] <= upper_limit)]

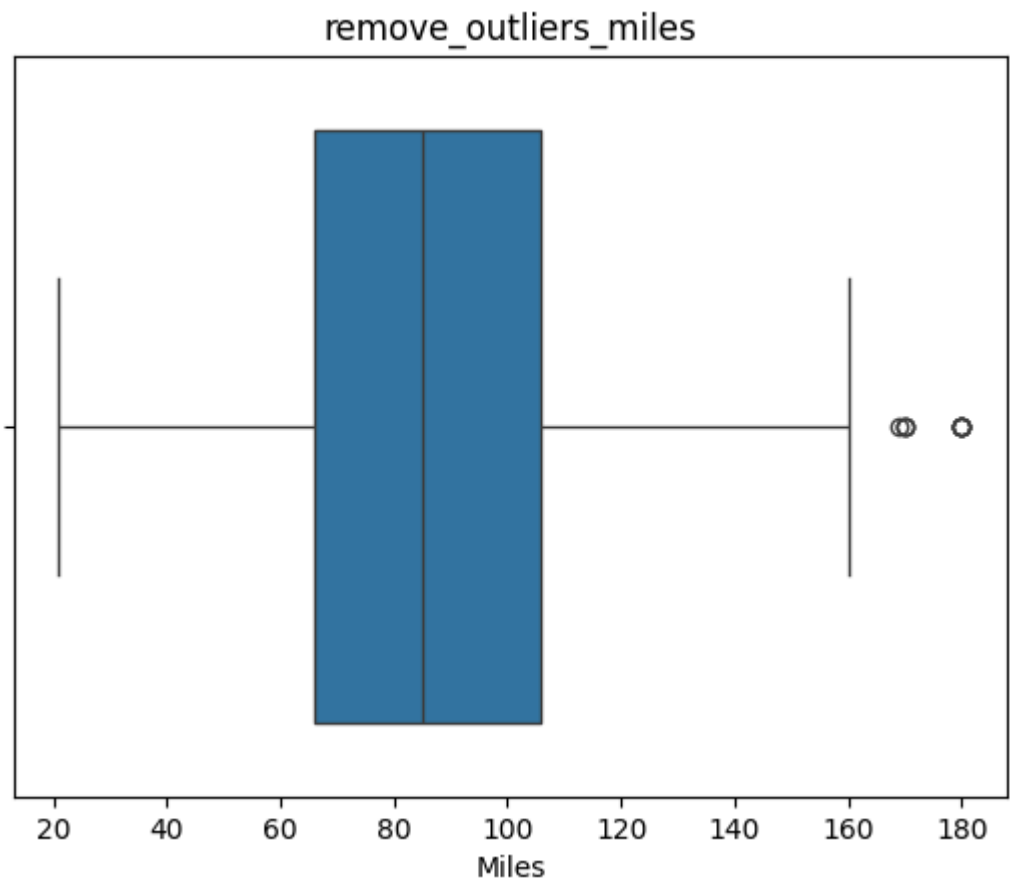
In [14]: plt.boxplot(data=data_no_outliers,x='Income', boxprops=dict(facecolor='Steelblue'), patch_artist=True)
plt.title('remove_outliers_income')
plt.ylabel('Income')
plt.show()
```



2.Miles

```
In [15]: Q1 = data['Miles'].quantile(0.25)
Q3 = data['Miles'].quantile(0.75)
IQR = Q3 - Q1
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
data_no_outliers = data[(data['Miles'] >= lower_limit) & (data['Miles'] <= upper_limit)]

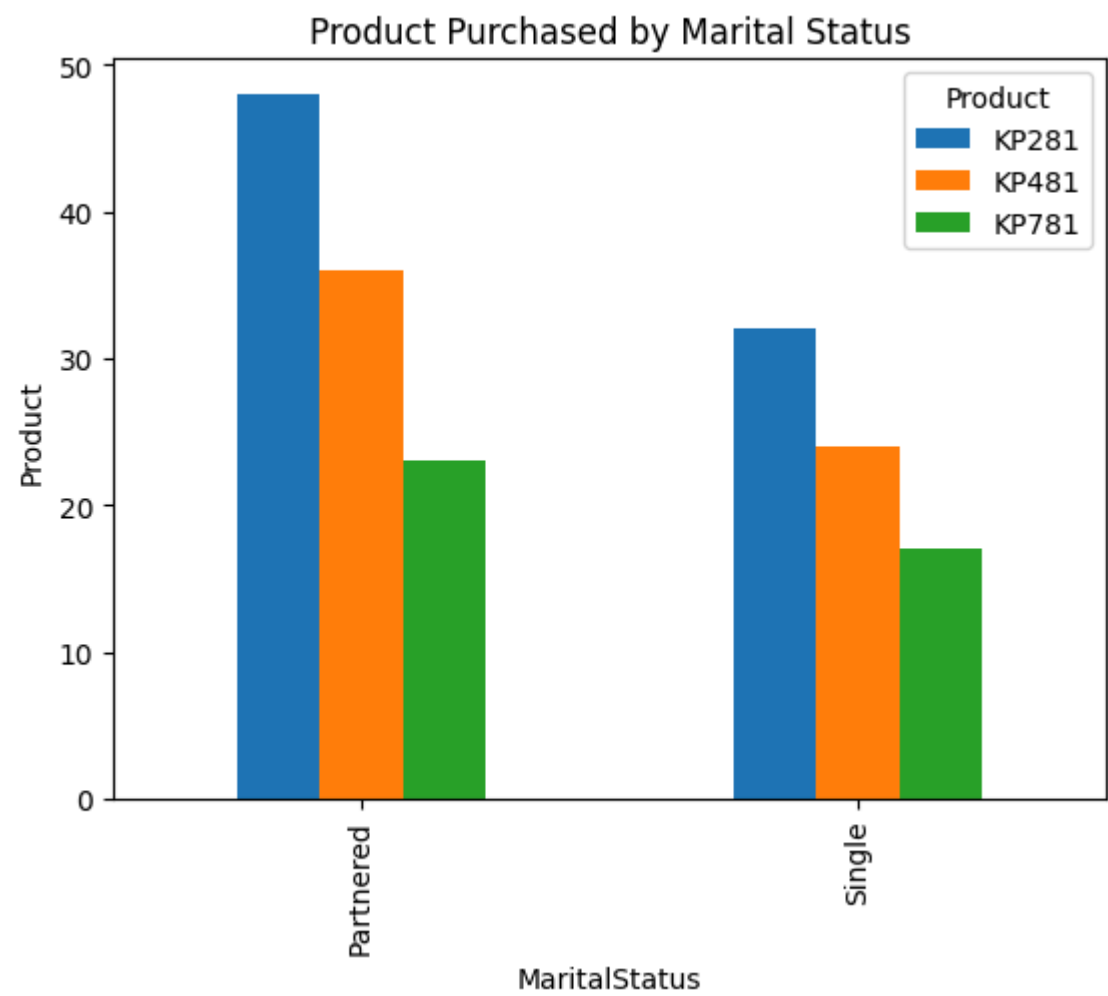
In [16]: sns.boxplot(data=data_no_outliers,x='Miles')
plt.title('remove_outliers_miles')
plt.show()
```



Step-5: Analytical Queries

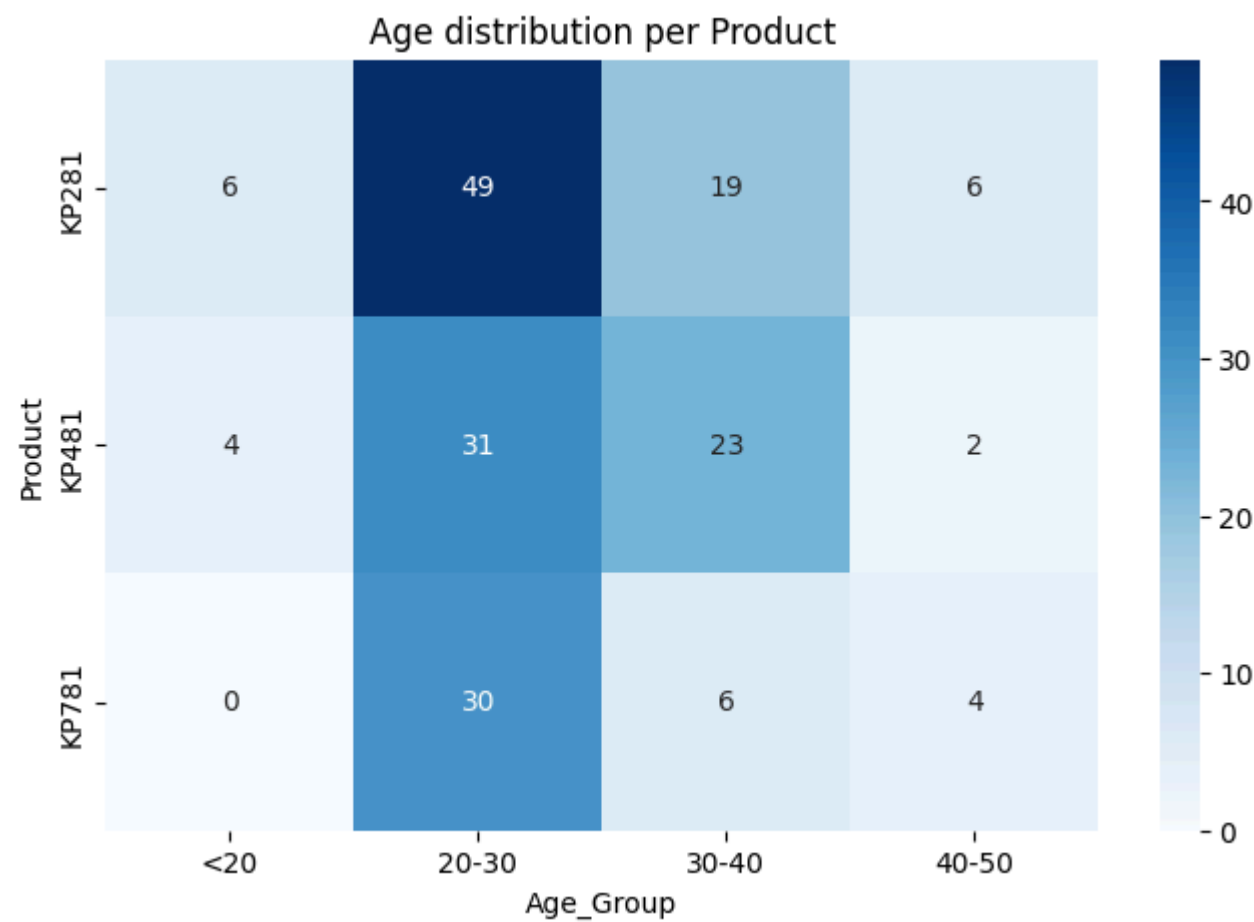
1.Product Purchased by Marital Status

```
In [17]: pd.crosstab(data['MaritalStatus'], data['Product']).plot(kind='bar')
plt.title("Product Purchased by Marital Status")
plt.ylabel("Product")
plt.show()
```



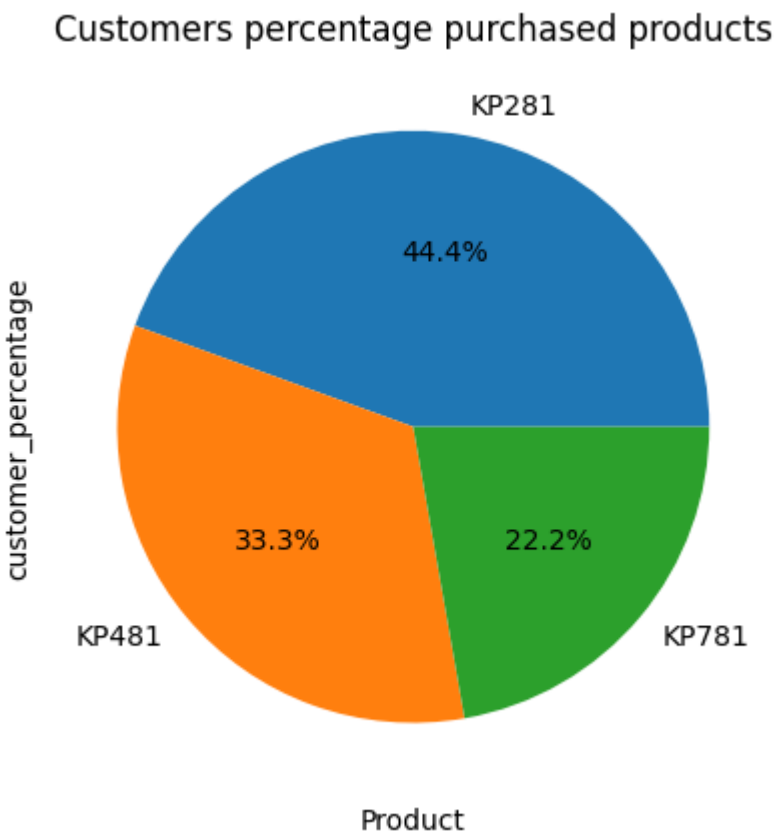
2.Age distribution per Product

```
In [18]: bins = [0, 20, 30, 40, 50, 60, 100]
labels = ['<20', '20-30', '30-40', '40-50', '50-60', '60+']
#Create Age_group
data['Age_Group'] = pd.cut(data['Age'], bins=bins, labels=labels)
ct_age = pd.crosstab(data['Product'], data['Age_Group'])
plt.figure(figsize=(8,5))
sns.heatmap(ct_age, annot=True, cmap='Blues')
plt.title("Age distribution per Product")
plt.show()
```



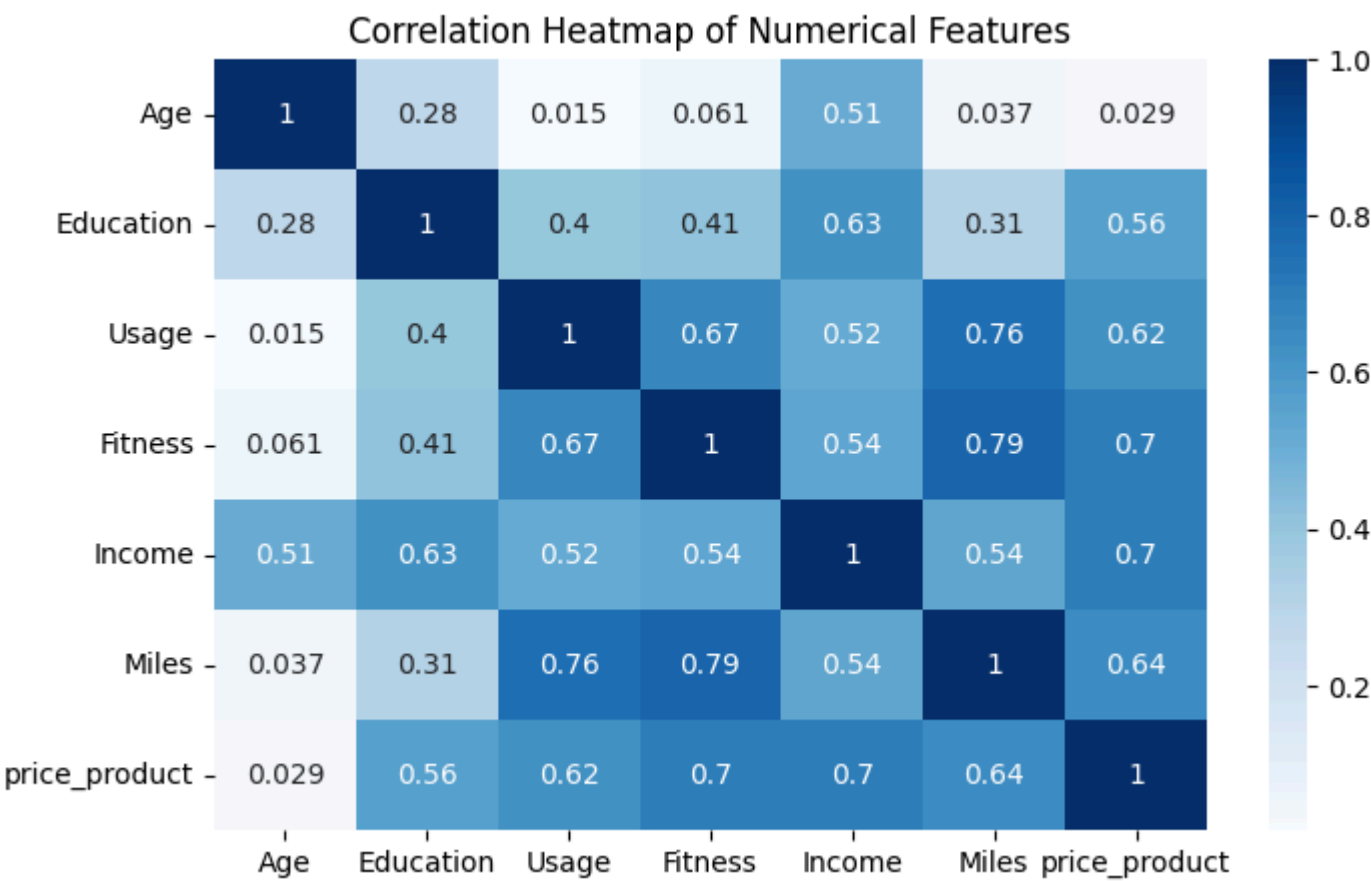
3.What is the Percentage of customers are Purchased Products

```
In [19]: no_of_custo=data['Product'].value_counts()
cust_per=round((no_of_custo/len(data))*100,2)
total=pd.DataFrame({'no_of_customers':no_of_custo,'customer_per':cust_per})
no_of_custo.plot(kind='pie', autopct='%1.1f%%')
plt.xlabel('Product')
plt.ylabel('customer_percentage')
plt.title('Customers percentage purchased products')
plt.show()
```



4.Check Correlation OF Numeric_Columns

```
In [20]: numeric_data = data.select_dtypes(include=['int64','float64'])
plt.figure(figsize=(8,5))
sns.heatmap(numeric_data.corr(), annot=True, cmap='Blues')
plt.title("Correlation Heatmap of Numerical Features")
plt.show()
```



5.Find The Probability of Male Customers are purchased the product of Kp781

Step 1: Identify the Relevant Columns

- Gender
- Product Purchased

Step 2: Filter the Dataset for Male Customers are purchased product Kp781

```
In [21]: male_data=data[data['Gender']=='Male']
total_males=len(male_data)
#to calculate product of male_Kp781
male_kp781 = male_data[male_data['Product'] == 'KP781']
male_kp781_count = len(male_kp781)
```

Step 3 : Probability of Male Customers are purchased product Kp781


```
In [22]: probability_of_males_KP781 = round(male_kp781_count / total_males,2)
print("probability_of_males are purchased KP781 is :",probability_of_males_KP781)
```

probability_of_males are purchased KP781 is : 0.32

Probability_of_males are purchased KP781 is 0.22

Step-6:Customer Profiling - Categorization of users.

```
In [23]: # Create Age Groups
data['Age_Group'] = pd.cut(data['Age'],
                           bins=[17, 30, 45, 60],
                           labels=['Young (18-30)', 'Middle Age (31-45)', 'Senior (46-60)'])

# Create Income Groups
data['Income_Level'] = pd.cut(data['Income'],
                              bins=[0, 30000, 60000, 100000],
                              labels=['Low Income', 'Medium Income', 'High Income'])

# Customer Profiling Table
profiling = pd.crosstab([data['Age_Group'], data['Income_Level'], data['Gender']],
                        data['Product'])

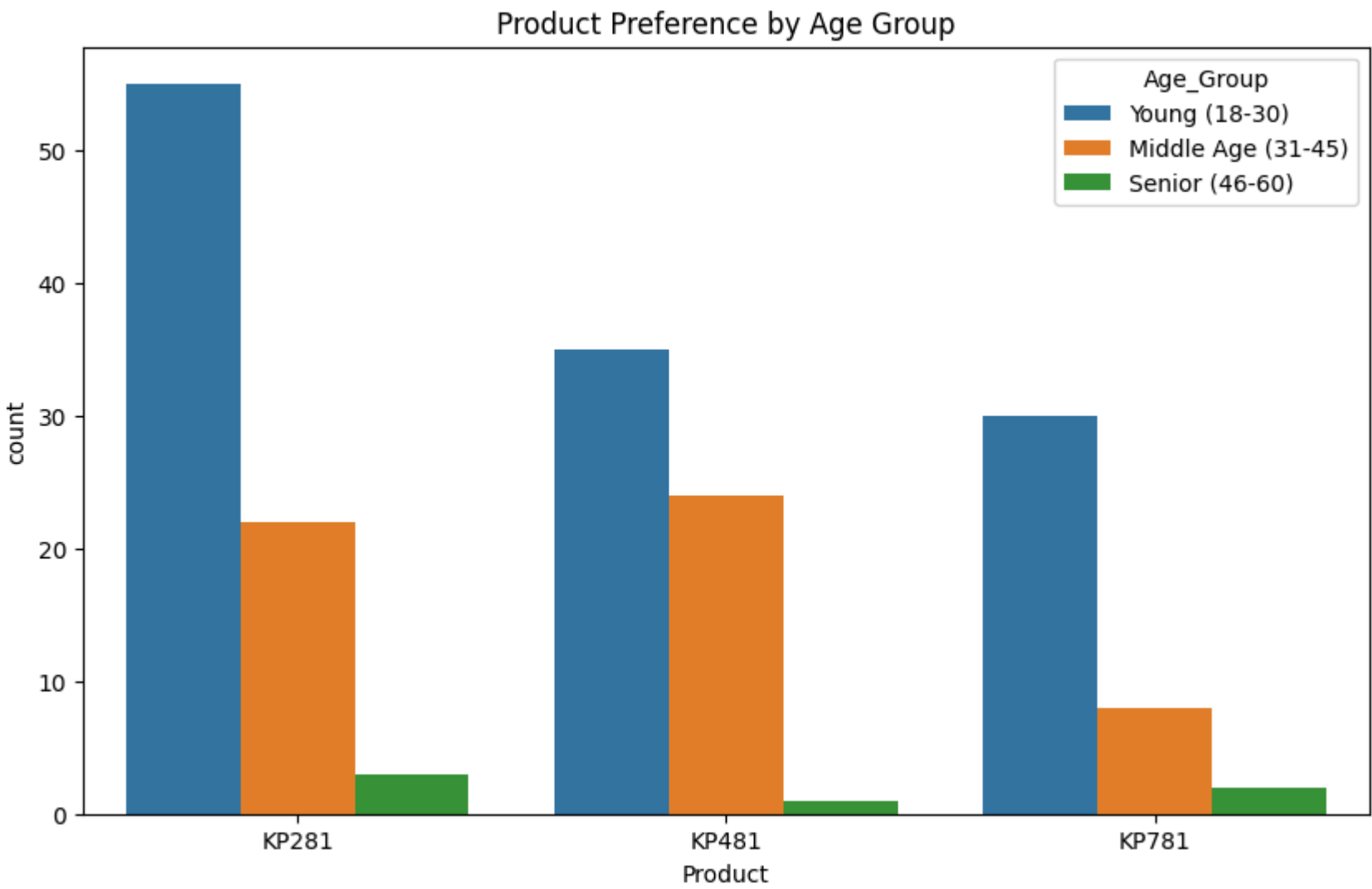
print("\n          --> Customer Profiling Table -->")
print(profiling)
```

--> Customer Profiling Table -->

Product			KP281	KP481	KP781
Age_Group	Income_Level	Gender			
Young (18-30)	Low Income	Male	1	0	0
		Female	28	17	2
	High Income	Male	25	18	9
		Female	0	0	4
		Male	1	0	14
Middle Age (31-45)	Medium Income	Female	8	8	0
		Male	11	9	0
	High Income	Female	2	4	1
		Male	1	3	6
		Male	1	1	0
Senior (46-60)	Medium Income	Male	1	1	0
	High Income	Female	2	0	0
		Male	0	0	1

(i).Which age group purchases the most treadmills across all product categories

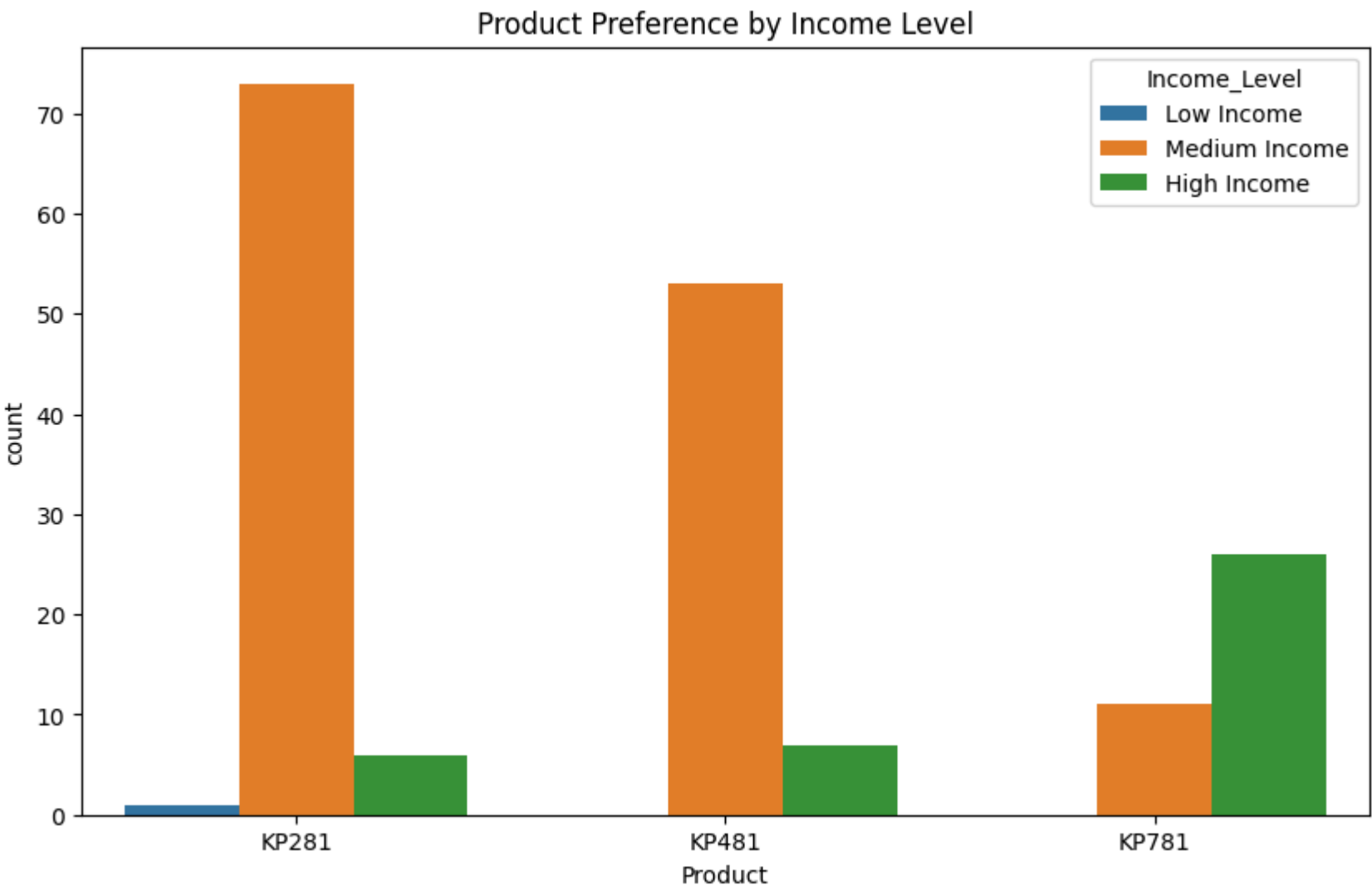
```
In [24]: plt.figure(figsize=(10,6))
sns.countplot(data=data, x='Product', hue='Age_Group')
plt.title("Product Preference by Age Group")
plt.show()
```



- **Young customers (18–30 years):** purchase the highest number of treadmills across all product categories. They show strong interest particularly in the KP281 and KP481 models.
- **The Middle Age group (31–45 years):**shows moderate purchase behavior, mainly choosing mid-range treadmills like KP481.
- **The Senior group (46–60 years):**Has the lowest number of buyers, indicating limited treadmill usage or a preference for alternative fitness options.

(ii).Which income category buy Products

```
In [25]: plt.figure(figsize=(10,6))
sns.countplot(data=data, x='Product', hue='Income_Level')
plt.title("Product Preference by Income Level")
plt.show()
```



- The **Medium Income** group purchases the highest number of treadmills across all product types, especially KP281 and KP481.
- **High Income** customers show strong preference for the premium KP781 model.
- **Low Income** customers make very minimal purchases overall.

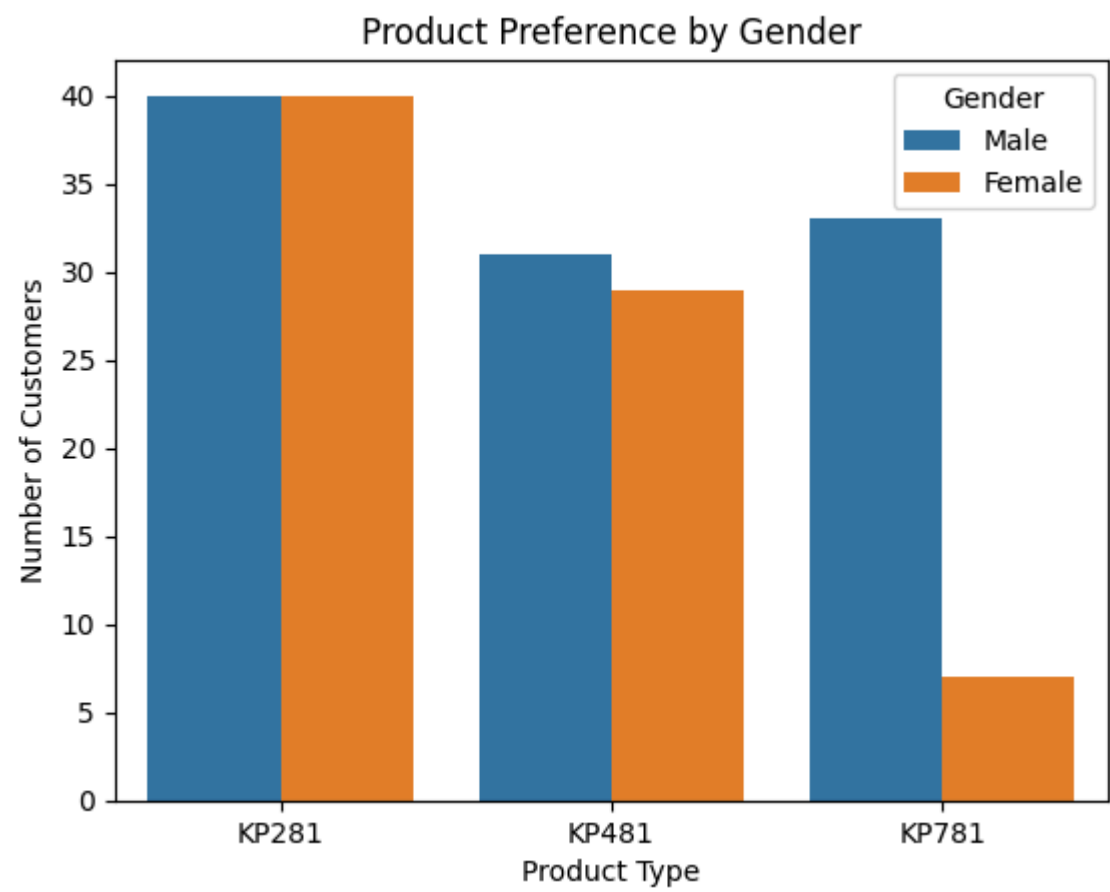
(iii).Does gender influence product choice?

```
In [26]: gender_product_ct = pd.crosstab(data['Gender'], data['Product'])
print("\n ---> Gender vs Product Preference Table--->")
print(gender_product_ct)

sns.countplot(data=data, x='Product', hue='Gender')
plt.title("Product Preference by Gender")
plt.xlabel("Product Type")
plt.ylabel("Number of Customers")
plt.show()
```

---> Gender vs Product Preference Table--->

Product	KP281	KP481	KP781
Gender			
Female	40	29	7
Male	40	31	33



- Both **males** and **females** equally prefer the KP281 model, showing that it is popular among all customer groups.
- However, **males** show a significantly higher preference for the premium model KP781, while females prefer the mid-range KP481 slightly more than males.

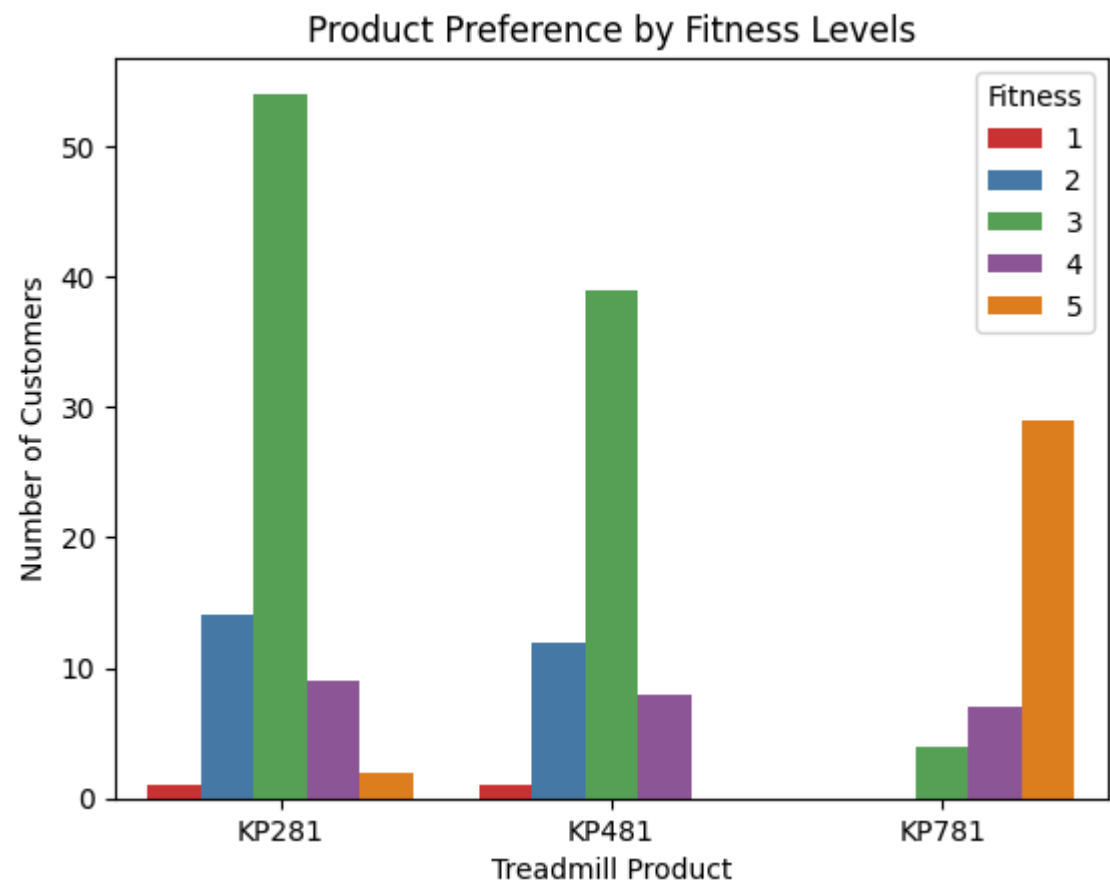
(iv).Fitness levels vs treadmill selection give code

```
In [27]: fitness_product_ct = pd.crosstab(data['Fitness'], data['Product'])
print("\n ---> Fitness Level vs Product Preference Table--->")
print(fitness_product_ct)

sns.countplot(data=data, x='Product', hue='Fitness', palette='Set1')
plt.title("Product Preference by Fitness Levels")
plt.xlabel("Treadmill Product")
plt.ylabel("Number of Customers")
plt.show()
```

---> Fitness Level vs Product Preference Table--->

Product	KP281	KP481	KP781
Fitness			
1	1	1	0
2	14	12	0
3	54	39	4
4	9	8	7
5	2	0	29



- Customers with higher fitness levels (Fitness 4 & 5) prefer the premium model KP781, showing they need advanced running features and better performance.
- Meanwhile, medium fitness users (Fitness 2 & 3) mostly buy KP281 and KP481, indicating affordability and basic workout needs.

```
In [28]: data.head()
```

```
Out[28]:
```

	Product	Age	Gender	Education	MaritalStatus	Usage	Fitness	Income	Miles	price_product	Age_Group	Income_Level
0	KP281	18	Male	14	Single	3	4	29562	112	1500	Young (18-30)	Low Income
1	KP281	19	Male	15	Single	2	3	31836	75	1500	Young (18-30)	Medium Income
2	KP281	19	Female	14	Partnered	4	3	30699	66	1500	Young (18-30)	Medium Income
3	KP281	19	Male	12	Single	3	3	32973	85	1500	Young (18-30)	Medium Income
4	KP281	20	Male	13	Partnered	4	2	35247	47	1500	Young (18-30)	Medium Income

Step-7:Probability

 **1. Marginal Probability :** Marginal probability refers to the probability of a single event happening without any condition.

What is the marginal probability that a customer runs more than 5 miles per week? Also, what is the conditional probability that a customer is high-income given that they ru

```
In [29]: total_customers = len(data)
miles_above_5 = len(data[data['Miles'] > 5])
marginal_prob_miles = miles_above_5 / total_customers
print("miles_above_5 :",miles_above_5)
print("Marginal Probability that a customer runs more than 5 miles per week:", marginal_prob_miles)
```

miles_above_5 : 180
Marginal Probability that a customer runs more than 5 miles per week: 1.0

 **2. Conditional Probability:** Conditional probability is the probability of an event occurring given that another event has already happened.

What is the conditional probability that a customer is high-income given that they run more than 5 miles per week?

```
In [30]: miles_group = data[data['Miles'] > 5]
high_income_miles = len(miles_group[miles_group['Income_Level'] == 'High'])
conditional_prob_income_given_miles = round(high_income_miles / miles_above_5,2)
print("high_income_miles is:",high_income_miles)
print("conditional_probablity that a customer runs more than 5 miles per week:",conditional_prob_income_given_miles)
```

high_income_miles is: 0
conditional_probablity that a customer runs more than 5 miles per week: 0.0

Step-8: Non-Graphical Analysis :- Value Counts and Unique attributes

a).Value Counts

The **value_counts()** function helps identify the most frequent occurrences of categorical data.

```
In [31]: data['Product'].value_counts()
```

```
Out[31]:
```

	count
Product	
KP281	80
KP481	60
KP781	40

dtype: int64

```
In [32]: data['Education'].value_counts()
```

Out[32]:

	count
Education	
16	85
14	55
18	23
15	5
13	5
12	3
21	3
20	1

dtype: int64

In [33]: data['MaritalStatus'].value_counts()

Out[33]:

	count
MaritalStatus	
Partnered	107
Single	73

dtype: int64

b).Unique_attributes

- The **nunique()** function tells how many unique values each column contains.

In [34]: data['Age'].unique()

Out[34]:

array([18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 43, 44, 46, 47, 50, 45, 48, 42])

In [35]: data['Miles'].unique()

Out[35]:

array([112, 75, 66, 85, 47, 141, 103, 94, 113, 38, 188, 56, 132, 169, 64, 53, 106, 95, 212, 42, 127, 74, 170, 21, 120, 200, 140, 100, 80, 160, 180, 240, 150, 300, 280, 260, 360])

In [36]: data['Usage'].unique()

Out[36]:

array([3, 2, 4, 5, 6, 7])

In [37]: data['Fitness'].unique()

Out[37]:

array([4, 3, 2, 1, 5])

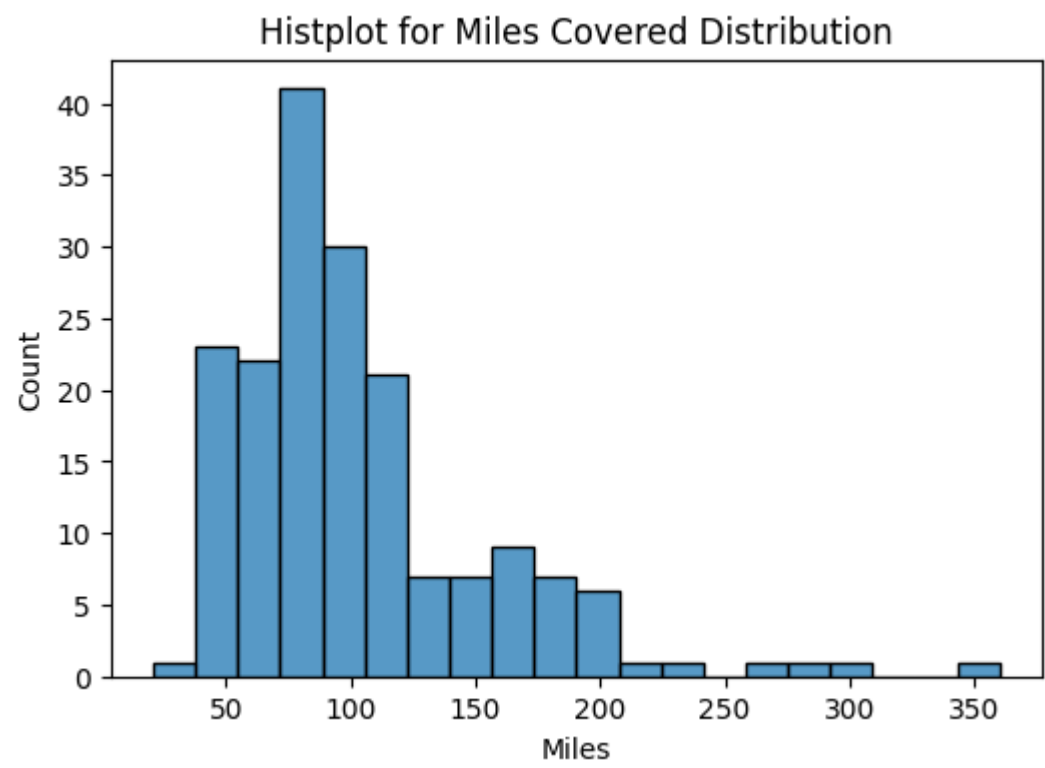
Step-9:Visual Analysis :- Univariate & Bivariate

a).Univariate Analysis:analyzing one variable at a time.

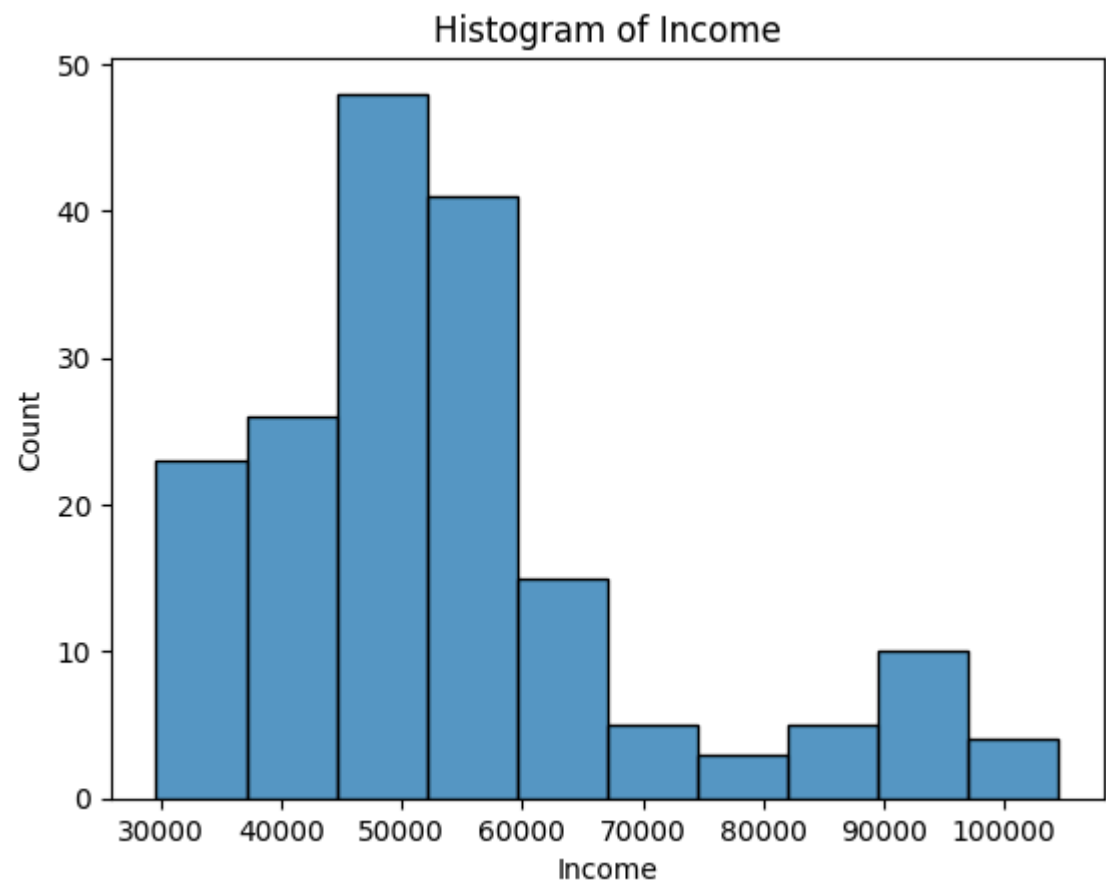
(i).Continuous Variables.

In [38]:

#Distplot for Miles
plt.figure(figsize=(6,4))
sns.histplot(data['Miles'], kde=False)
plt.title("Histplot for Miles Covered Distribution")
plt.show()

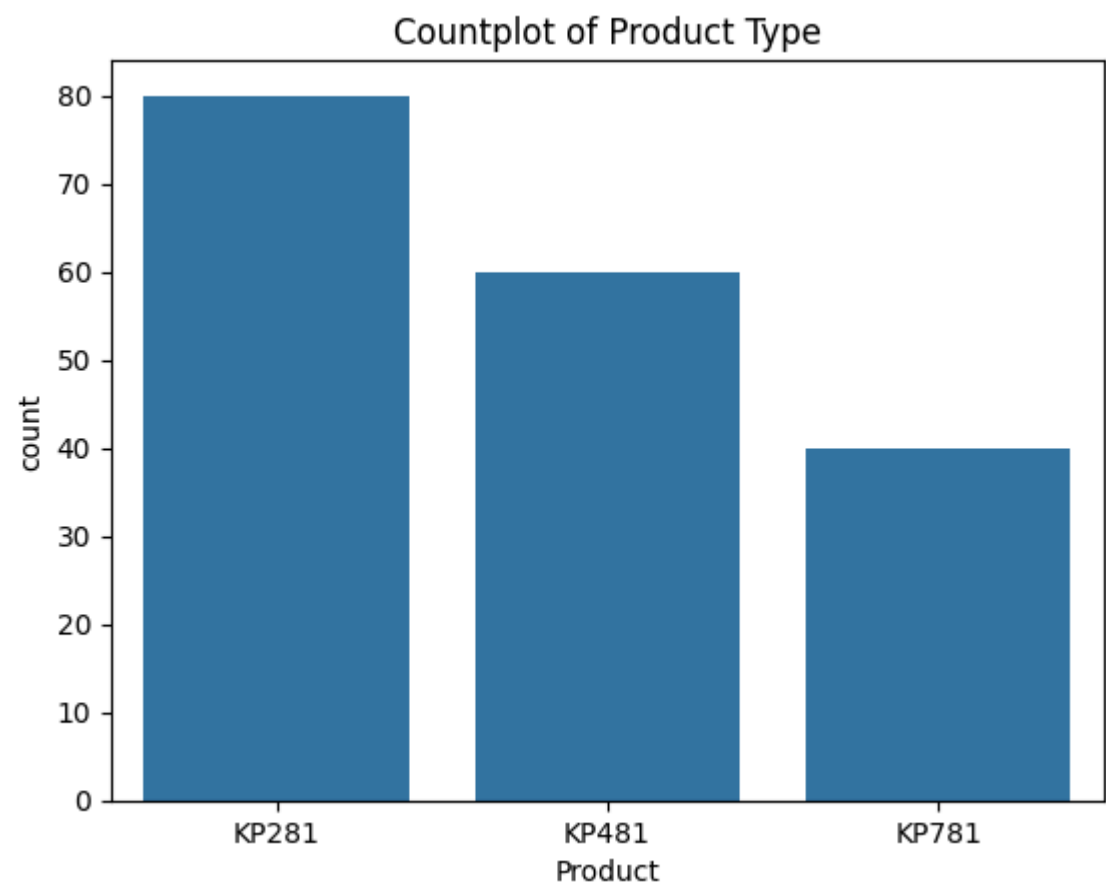


```
In [39]: #Histplot For Income
sns.histplot(data["Income"], bins=10)
plt.title("Histogram of Income")
plt.show()
```



(ii).Categorical Variable

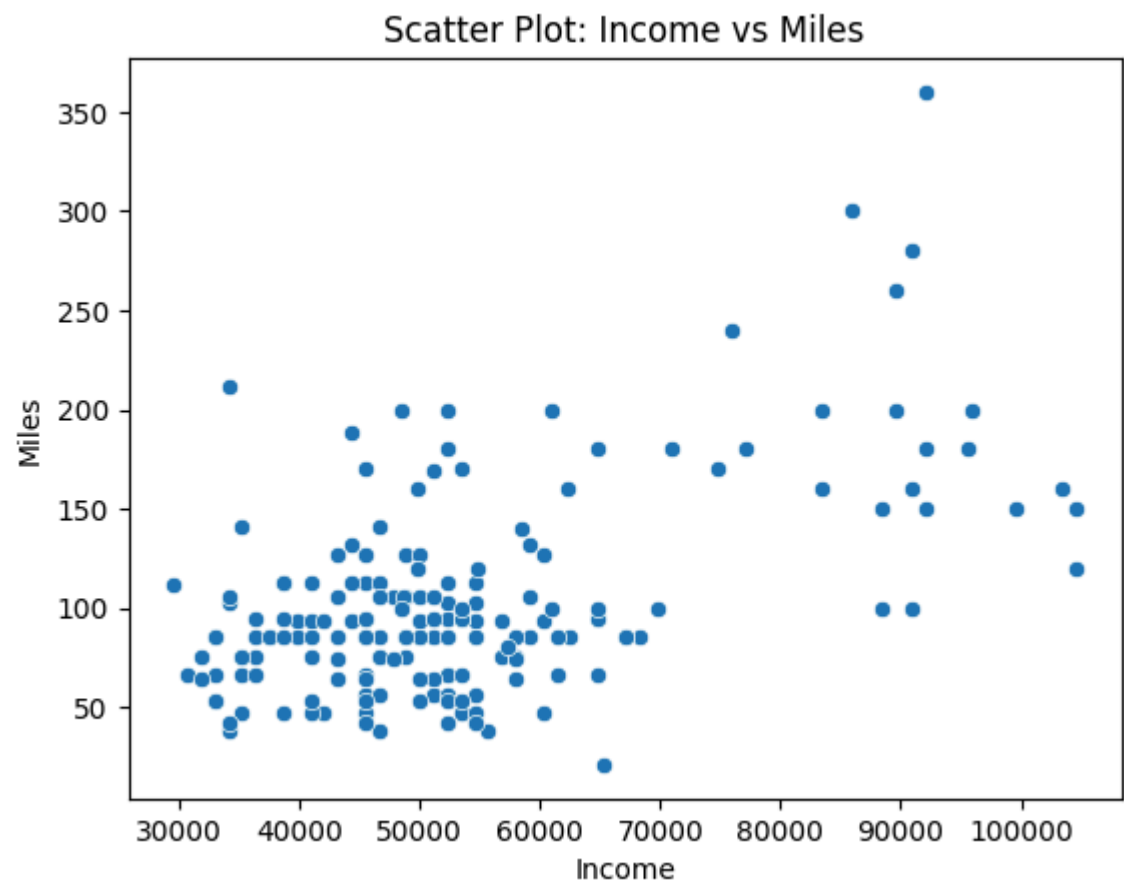
```
In [40]: #Countplot For Product
sns.countplot(x=data["Product"])
plt.title("Countplot of Product Type")
plt.show()
```



b).Bivariate Analysis:analyzing two variables together.

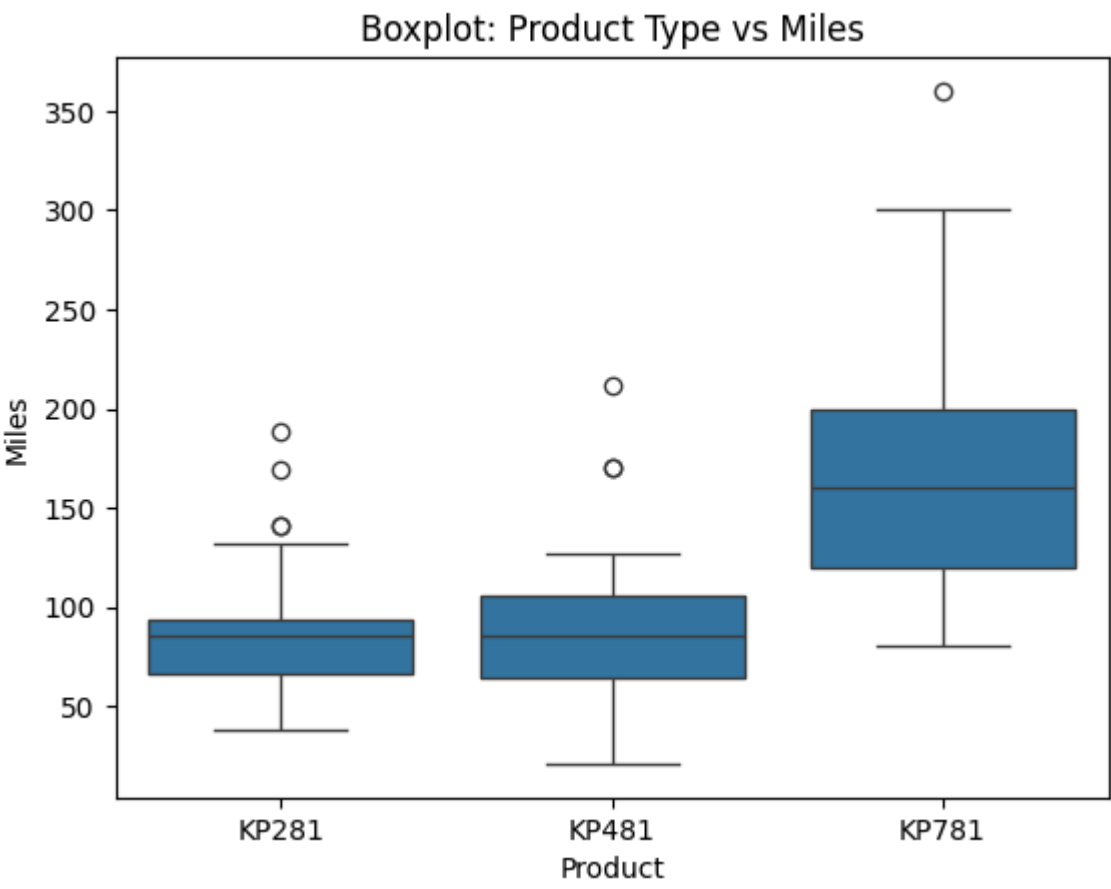
(i).For continuous variables:

```
In [41]: #Scatter plot for Income Vs Miles
sns.scatterplot(x=data["Income"], y=data["Miles"])
plt.title("Scatter Plot: Income vs Miles")
plt.show()
```



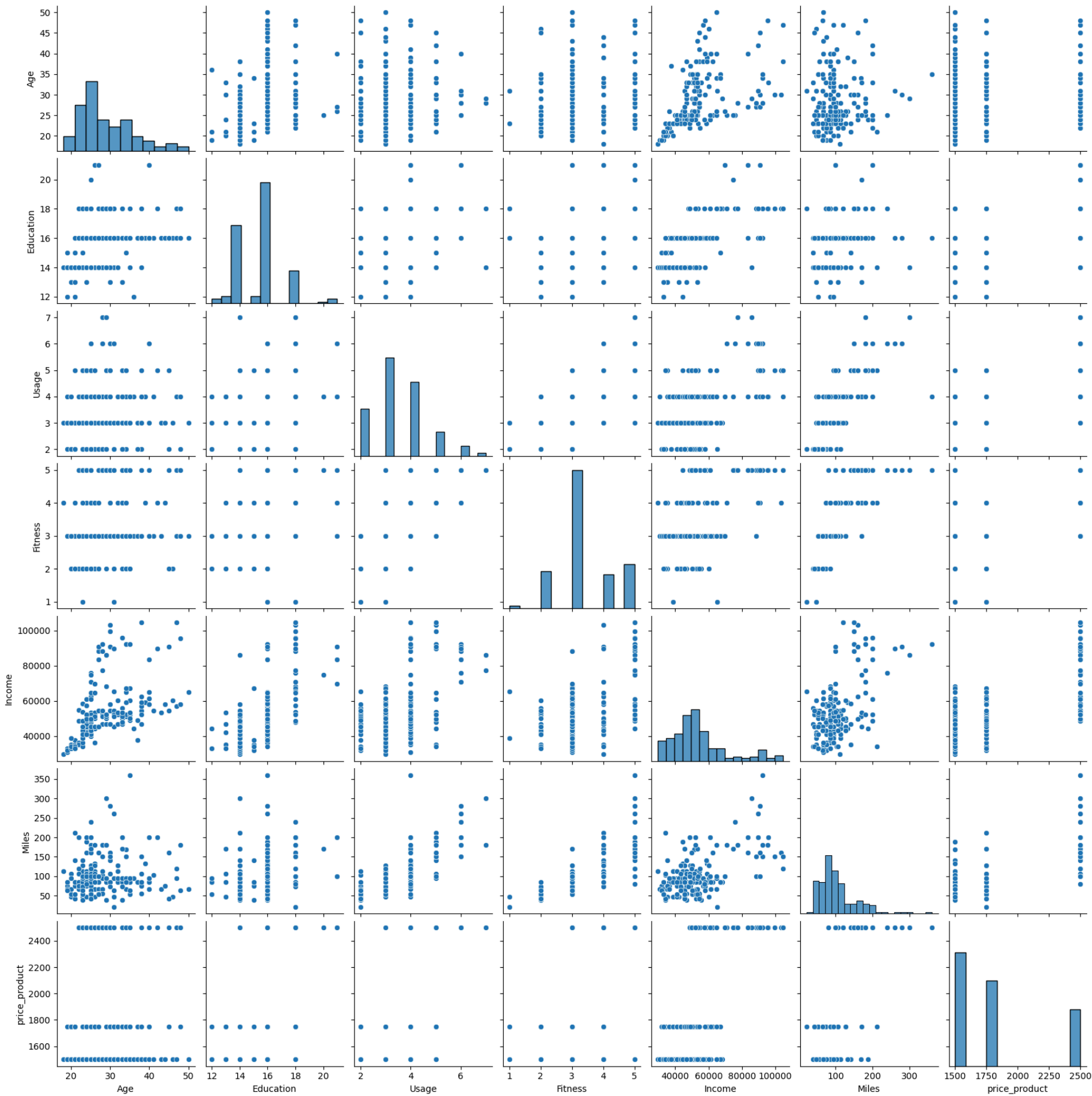
(ii).Categorical vs Continuous

```
In [42]: #Box plot Product Type vs Miles
sns.boxplot(x=data["Product"], y=data["Miles"])
plt.title("Boxplot: Product Type vs Miles")
plt.show()
```



(iii).Correlation Analysis

```
In [43]: sns.pairplot(data)
plt.show()
```



Business Insights:

✦ Comments on the Range of Attributes:

- The dataset contains demographic and usage-based attributes such as age, gender, marital status, income, product type, and treadmill usage.
- The values vary across a wide range, showing diverse customer profiles.

✦ Comments on the Distribution & Relationships:

- Most customers fall in a younger age group with mid-level income. KP281 is the most purchased model.
- A slight positive relationship exists between income and treadmill usage, meaning higher-income users tend to use the treadmill more.

✦ Comments on Univariate & Bivariate Plots:

- Univariate plots: Show product preference, income spread, and usage variation among users.
- Bivariate plots: Show that treadmill usage varies across product types, and income shows a mild association with miles covered.

✦ Comments on Customer Profiling:

- Young customers (18–30) are the top buyers across all models.
- Medium-income customers purchase the most, mainly KP281 & KP481.
- High-income and high-fitness users prefer the premium KP781.
- Males show more interest in advanced models, while females prefer mid-range options.
- Senior age group participates less → needs more targeted marketing.

Recommendations:

- Promote KP781 model to high-income customers, as they show higher usage and interest based on analysis.
- Introduce EMI or discount offers for mid-income customers to increase sales of premium treadmill models.
- Offer personalized workout plans or rewards for high usage to improve customer engagement and brand loyalty.
- Improve marketing focus on married and partnered customers, as they appear as majority buyers.
- Develop a fitness tracking mobile app to increase engagement and attract technology-oriented buyers.
- Create bundles or upgrade options (ex: accessories + maintenance plan) for frequently using treadmill customers.
- Target young age groups (18–30) on social platforms such as Instagram and YouTube because they form a strong buyer base.
- Increase after-sales support response time, since consistent treadmill usage suggests maintenance expectations.
- Focus marketing on young and medium-income buyers, highlight premium features for high-income fit users, and create targeted campaigns to engage seniors.

```
In [44]: !jupyter nbconvert --execute --to html ""
```